

Architecture.md v1.1

Chapter 2 — Design Philosophy & Architectural Principles

2.1 Purpose of this Architecture

The purpose of this architecture is **not** simply to build another AI assistant.

The objective is to build a long-term **AI Operating System (AI OS)** where intelligent components work together through a structured architecture.

Every future feature, module, agent, or AI model must fit into this architecture rather than modifying it.

This architecture is intended to remain valid for multiple years of development.

2.2 Architectural Philosophy

AI OS follows five fundamental philosophies.

Philosophy 1 — AI OS is an Operating System

Jarvis is **not** the project.

Friday is **not** the project.

Both are applications running inside AI OS.

AI OS is the platform responsible for:

- Memory
- Communication
- Agent Management
- Permissions
- Logging
- Configuration
- Storage
- Future Expansion

Removing Jarvis should not break AI OS.

Replacing Friday should not require redesign.

Changing AI models should not affect the architecture.

Philosophy 2 — Personalities Think, Agents Work

The personalities never perform business logic.

Their responsibilities are:

- Understanding the user
- Planning
- Decision making
- Delegating work
- Explaining results

Worker agents perform all specialized work.

Example:

User:

Analyze my Instagram account.

Incorrect:

Jarvis performs analytics.

Correct:

Jarvis

↓

Agent Manager

↓

Analytics Agent

↓

Agent Manager

↓

Jarvis

↓

User

The personalities are coordinators, not specialists.

Philosophy 3 — One Source of Truth

There shall only be one authoritative source for:

Memory

One shared memory.

Never separate memories.

Configuration

One configuration system.

Never duplicate configuration.

Permissions

One permission system.

Never define permissions inside individual agents.

User Identity

One user profile.

Every module references the same identity.

Architecture

This document (Architecture.md) is the technical source of truth.

No implementation may contradict it without an approved revision.

Philosophy 4 — Loose Coupling

Every module must remain independent.

Changing one module should not require editing unrelated modules.

Example:

Replacing:

Analytics Agent

must not affect:

- Business Agent
- Trading Agent
- Planner
- Memory
- Jarvis

Modules communicate through contracts, never through internal implementation details.

Philosophy 5 — Long-Term Maintainability

Every architectural decision should optimize for:

- Readability
- Simplicity
- Testability
- Modularity
- Future expansion

Never optimize solely for writing less code today.

2.3 Single Responsibility Principle

Every major component owns exactly one responsibility.

Examples:

Router

Responsible for:

- Intent routing

Not responsible for:

- Memory
 - AI reasoning
 - Database
-

Jarvis

Responsible for:

- Strategic reasoning
- Planning
- Delegation

Not responsible for:

- Analytics
 - Database
 - Trading
 - Browser automation
-

Analytics Agent

Responsible for:

- Analytics only

Not responsible for:

- Scheduling
- Business
- Research

Every module should answer one question:

"What is my only responsibility?"

2.4 Dependency Direction

Dependencies always flow downward.

User

↓

Router

↓

Jarvis / Friday

↓

Agent Manager

↓

Worker Agents

↓

Core Services

↓

Persistence

Lower layers never know who called them.

Example:

Analytics Agent does not know whether:

- Jarvis requested it
- Friday requested it

It simply executes the assigned task.

2.5 Centralized Decision Making

Only three components may make architectural decisions during runtime.

Router

Decides:

Which personality receives the request.

Jarvis / Friday

Decide:

What work needs to be done.

Agent Manager

Decides:

Which worker agents execute.

No other component makes routing decisions.

2.6 Shared Memory Philosophy

Memory belongs to AI OS.

Not to Jarvis.

Not to Friday.

Not to worker agents.

Memory exists independently.

If Jarvis is replaced tomorrow:

Memory remains.

If Friday is disabled:

Memory remains.

If AI models change:

Memory remains.

2.7 Extensibility Philosophy

Adding a new feature should require:

1. Creating a new module.
2. Registering it.
3. Using it.

No existing module should require modification.

Example:

Adding:

Music Agent

Should not require editing:

- Planner
- Analytics
- Trading
- Business
- Jarvis
- Friday

Only registration should be necessary.

2.8 Human-in-the-Loop Principle

AI OS assists.

The user decides.

Critical actions require explicit confirmation.

Examples include:

- File deletion
- Financial actions
- Trading execution
- System shutdown
- Permanent memory deletion

No module may bypass user approval.

2.9 Fail-Safe Design

Every failure should degrade gracefully.

Preferred behavior:

- Return structured errors.
- Preserve user data.
- Log the event.
- Continue operating where possible.

The system should never crash because one worker agent fails.

2.10 Version Evolution

Architecture evolves through versioning.

Major changes require:

- New architecture version
- Documented change log
- Updated specifications

Older modules remain compatible whenever practical.

2.11 Definition of Success

A feature is considered architecturally complete only if it:

- Follows this architecture.
- Uses shared services correctly.

- Avoids duplicate responsibilities.
 - Is independently testable.
 - Can be replaced without affecting unrelated modules.
 - Requires no undocumented assumptions.
-

End of Chapter 2

Next Chapter: Chapter 3 — Core Components (Router, Jarvis, Friday, Agent Manager, Memory Service)